Dataset Pipeline - Travel Sentence Classifier

Ce document décrit la stratégie complète de données pour le projet de classification de phrases de voyage.

---

1. Data Objective

Objectif Principal

Le système doit classifier les phrases des utilisateurs liées aux voyages en trois catégories :

- **VALID** : Phrases qui expriment explicitement une demande d'itinéraire ou de trajet entre deux lieux
- **INVALID** : Phrases qui ne sont pas liées à une demande d'itinéraire
- **AMBIGU** : Phrases qui mentionnent des lieux ou des mouvements mais sans intention claire de demander un itinéraire

Approche Progressive

Nous commençons par une **classification binaire (VALID vs INVALID)** pour construire un baseline solide avant d'étendre le modèle à trois classes. Cette approche permet de :

- Valider la faisabilité du problème
- Établir des métriques de référence
- Identifier les patterns discriminants
- Construire une base robuste avant d'ajouter la complexité de l'ambiguïté

Une fois le modèle binaire validé avec de bonnes performances, nous introduirons la classe **AMBIGU** pour améliorer la robustesse du système.

---

2. Dataset Creation Strategy

Objectif de Volume

Générer un dataset très large de **50,000 à 100,000 phrases** pour garantir une diversité suffisante et permettre l'entraînement de modèles robustes.

Règles de Génération

Diversité Linguistique Maximale

- **Variations grammaticales** : Déclaratif, interrogatif, impératif, conditionnel
- **Niveaux de formalité** : Langage soutenu, familier, SMS, argot
- **Longueurs variables** : Phrases courtes (2-3 mots) à longues (20+ mots)
- **Ordres de mots** : Structures syntaxiques variées
- **Ellipses** : Phrases incomplètes mais compréhensibles

Simulation Réaliste des Entrées Utilisateur

- **Fautes d'orthographe** : Erreurs courantes sur les noms de villes
  - Exemples : "pariis", "toulousse", "bordo", "marseile"
- **Langage SMS/Shorthand** : Abréviations et contractions
  - Exemples : "jveux", "chui", "stp", "svp", "depuis" → "depui"
- **Typage rapide** : Caractères manquants, doublons accidentels
- **Ponctuation variable** : Absence de ponctuation, ponctuation excessive, emojis

Formes Interrogatives et Impératives

- Questions directes : "Comment aller de X à Y ?"
- Questions indirectes : "Peux-tu me dire comment..."
- Impératifs : "Donne-moi un trajet...", "Trouve-moi..."

- Formulations polies : "J'aimerais...", "Je souhaiterais..."

Bruit Synthétique

- Phrases tronquées : "je veux", "aller", "train"
- Caractères aléatoires : "asdadadasd", "mdrrr t ki"
- Texte absurde : "train machin truc ??"
- Expressions hors contexte : "Je mange une pomme"

Distribution des Classes

Phase Binaire (Baseline) :

- **70% INVALID** : Prévenir la sur-prédiction de VALID, refléter la distribution réelle
- **30% VALID** : Suffisant pour apprendre les patterns de requêtes valides

Phase 3 Classes (Finale) :

- **60% INVALID**
- **30% VALID**
- **10% AMBIGU**

Design des Phrases VALID

Les phrases VALID doivent contenir :

- **Deux lieux distincts** (origine et destination)
- **Intention explicite** de demander un itinéraire/trajet
- **Mots-clés de mouvement** : "aller", "trajet", "depuis", "vers", "itinéraire", "billet", "train"

Exemples de structures VALID :

- "Je veux aller de [ville1] à [ville2]"
- "Trajet [ville1] → [ville2]"
- "Comment me rendre à [ville2] depuis [ville1] ?"
- "[ville1] [ville2] svp"
- "jveux un train pour [ville2] depuis [ville1]"

Design des Phrases INVALID

Les phrases INVALID peuvent être :

- **Sans mention de trajet** : Questions générales, conversations, requêtes non liées
- **Avec villes mais sans intention** : "Paris est magnifique", "Je suis à Lyon"
- **Incomplètes** : Phrases tronquées sans contexte
- **Nonsense** : Texte aléatoire, caractères sans sens
- **Erreurs structurelles** : "je veux [ville]" (une seule ville), "trajet [ville]" (destination manquante)

Exemples de structures INVALID :

- "Je mange une pomme"
- "Paris est magnifique la nuit"
- "je veux aller à Paris" (pas d'origine)
- "trajet" (incomplet)
- "asdadadasd" (nonsense)

---

3. Preprocessing

Normalisation du Texte

Le preprocessing doit rester **minimal** pour préserver la diversité du monde réel et ne pas masquer les patterns importants (fautes, variations, etc.).

Étapes de Normalisation

1. **Lowercase** : Conversion en minuscules
   - Raison : Uniformiser sans perdre d'information
   - Exception : Peut être optionnel si on veut préserver les majuscules pour les noms propres
2. **Nettoyage des espaces multiples** : Remplacer `\s+` par un seul espace
   - Raison : Normaliser les espaces sans altérer le contenu
3. **Normalisation minimale de la ponctuation** :
   - Supprimer les espaces en début/fin
   - Conserver la ponctuation originale (points, virgules, points d'interrogation)
   - Raison : La ponctuation peut être informative (questions vs affirmations)

Ce qu'on NE fait PAS

- ❌ Correction des fautes d'orthographe
- ❌ Suppression des emojis (peuvent être informatifs)
- ❌ Normalisation des accents (peut masquer des patterns régionaux)
- ❌ Stemming ou lemmatisation (perd de l'information)
- ❌ Suppression des stop words (peuvent être discriminants : "depuis", "vers")

Split Train/Validation/Test

Distribution recommandée :

- **Train** : 70% (ou 80% pour datasets plus petits)
- **Validation** : 15% (ou 10%)
- **Test** : 15% (ou 10%)

Règles importantes :

- **Stratification** : Préserver la distribution des classes dans chaque split
- **Random seed fixe** : Pour reproductibilité
- **Pas de leakage** : S'assurer qu'aucune phrase du train n'apparaît dans validation/test
- **Validation croisée** : Optionnel pour datasets plus petits (< 10k)

---

4. Vectorization (TF-IDF)

Choix de TF-IDF pour les Modèles Baseline

TF-IDF (Term Frequency-Inverse Document Frequency) est choisi pour les modèles baseline car :

- ✅ **Robustesse** : Gère bien les variations de vocabulaire
- ✅ **Coût computationnel faible** : Rapide à entraîner et à utiliser
- ✅ **Interprétabilité** : Permet d'analyser l'importance des mots
- ✅ **Excellent pour baselines** : Établit une référence solide avant d'utiliser des embeddings

Paramètres Recommandés

```
TfidfVectorizer(
    ngram_range=(1, 2),       # Unigrams et bigrams
    max_features=50000,        # Limiter pour éviter la surcharge mémoire
    min_df=2,                 # Mot doit apparaître au moins 2 fois
    max_df=0.95,              # Ignorer les mots trop fréquents (stop words)
    lowercase=True,           # Normalisation en minuscules
```

```
    strip_accents=None       # Conserver les accents
)
```

Justification des paramètres :

- **ngram_range=(1, 2)** : Capture les mots individuels et les paires de mots (ex: "de Paris", "vers Lyon")
- **max_features=50000** : Équilibre entre couverture et performance
- **min_df=2** : Élimine les mots trop rares (probablement du bruit)
- **max_df=0.95** : Élimine les mots présents dans presque tous les documents (peu discriminants)

---

5. Baseline Models (Binary Classification)

Les Trois Modèles à Tester

1. Logistic Regression (Recommandé)

Avantages :

- ✅ Interprétabilité : Coefficients directement interprétables
- ✅ Probabilités calibrées : Utiles pour détecter l'ambiguïté
- ✅ Rapide à entraîner et prédire
- ✅ Gère bien les features creuses (TF-IDF)
- ✅ Régularisation efficace (L1/L2)

Hyperparamètres clés :

- `C` : Force de régularisation (inverse de lambda)
- `solver` : 'lbfgs' ou 'liblinear' pour datasets moyens
- `class_weight` : 'balanced' pour gérer le déséquilibre

Quand l'utiliser : Modèle principal pour baseline, excellent point de départ.

2. Linear SVM (LinearSVC)

Avantages :

- ✅ Bonne performance sur données haute dimension
- ✅ Résistant au bruit
- ✅ Rapide avec kernel linéaire
- ✅ Marges larges pour séparation

Inconvénients :

- ❌ Pas de probabilités natives (nécessite calibration)
- ❌ Moins interprétable que Logistic Regression

Quand l'utiliser : Alternative si Logistic Regression ne performe pas assez bien.

3. Naive Bayes (Baseline Minimum)

Avantages :

- ✅ Très rapide
- ✅ Simple et léger
- ✅ Bon baseline de référence
- ✅ Gère bien les features creuses

Inconvénients :

- ❌ Hypothèse d'indépendance forte (rarement vraie)
- ❌ Moins performant que les autres modèles

Quand l'utiliser : Baseline minimum pour comparaison, modèle très simple.

Pourquoi Tester les Trois

1. **Comparaison de performance** : Identifier le meilleur modèle pour notre cas
2. **Validation de l'approche** : Si tous performant mal, problème de features/données
3. **Trade-offs** : Vitesse vs performance vs interprétabilité
4. **Robustesse** : Vérifier que les résultats sont cohérents

Métriques à Collecter

Métriques Globales

- **Accuracy** : Taux de classification correcte global
- **F1-Score (macro)** : Moyenne harmonique de précision et rappel (équilibré)
- **F1-Score (micro)** : F1 calculé sur toutes les prédictions
- **F1-Score (weighted)** : F1 pondéré par la fréquence des classes

Métriques par Classe

- **Precision** : Pourcentage de prédictions correctes parmi les prédictions positives
- **Recall** : Pourcentage de vrais positifs détectés
- **F1-Score** : Moyenne harmonique de précision et rappel

Métriques Spéciales

- **Recall pour INVALID** : Critique pour éviter de laisser passer du bruit
- **Precision pour VALID** : Critique pour éviter les faux positifs
- **Matrice de confusion** : Visualisation des erreurs par classe

Métriques d'Ambiguïté (Phase 2)

- **Zone d'ambiguïté** : Pourcentage de phrases dans la zone [0.40, 0.60]
- **Calibration des probabilités** : Brier score, calibration curve

---

6. Adding the AMBIGU Class

Quand Ajouter l'Ambiguïté

L'ajout de la classe AMBIGU se fait **après validation du modèle binaire** lorsque :

- ✅ Le modèle binaire atteint de bonnes performances (> 85% F1)
- ✅ Les patterns VALID/INVALID sont bien appris
- ✅ On veut améliorer la robustesse du système
- ✅ On veut gérer les cas limites de manière élégante

Définition de l'Ambiguïté

Une phrase est **AMBIGU** si elle :

- Mentionne des lieux ou du mouvement
- N'exprime **pas clairement** une demande d'itinéraire
- Pourrait être interprétée comme VALID ou INVALID selon le contexte
- Nécessite une clarification pour être classée avec certitude

Exemples de Phrases AMBIGU

- "Je vais bientôt partir à Paris rejoindre ma mère qui arrive de Toulouse."
    - Mentionne deux villes mais c'est une déclaration, pas une demande
- "Je viens d'arriver à Marseille depuis Bordeaux."
    - Décrit un mouvement passé, pas une demande future

- "Je dois aller à Lyon demain."
  - Mentionne une destination mais pas d'origine claire
- "Je pars de Nice."
  - Mentionne une origine mais pas de destination
- "Direction Paris ?"
  - Question trop vague, pourrait être une demande ou une confirmation
- "Je veux voyager mais je ne sais pas où."
  - Intention de voyage mais pas de lieux spécifiques

Règles de Labeling AMBIGU

1. **Deux lieux mais pas de demande explicite** → AMBIGU
2. **Un seul lieu mentionné** → AMBIGU (sauf si contexte très clair)
3. **Mouvement décrit mais pas demandé** → AMBIGU
4. **Question vague sur un lieu** → AMBIGU
5. **Phrase incomplète avec contexte de voyage** → AMBIGU

Distribution Recommandée (3 Classes)

- **60% INVALID** : Reste majoritaire pour prévenir sur-prédiction
- **30% VALID** : Suffisant pour apprendre les patterns
- **10% AMBIGU** : Minoritaire mais important pour robustesse

Cette distribution reflète mieux la réalité : la plupart des entrées ne sont pas des requêtes, et une petite portion est ambiguë.

---

7. Automatic Ambiguity Detection

Utilisation des Probabilités de Logistic Regression

Les probabilités de sortie de la Logistic Regression peuvent être utilisées pour **bootstrap** (détecter automatiquement) les échantillons ambigus.

Seuils de Probabilité

Règle de décision basée sur les probabilités :

- **VALID** si `P(VALID) > 0.60` : Confiance élevée en VALID
- **INVALID** si `P(VALID) < 0.40` : Confiance élevée en INVALID
- **AMBIGU** si `0.40 ≤ P(VALID) ≤ 0.60` : Zone d'incertitude

Processus de Bootstrap

1. **Entraîner le modèle binaire** sur VALID/INVALID
2. **Prédire les probabilités** sur un dataset non labelé ou de validation
3. **Identifier les phrases dans la zone [0.40, 0.60]**
4. **Examiner manuellement** un échantillon pour valider qu'ils sont effectivement ambigus
5. **Labeler comme AMBIGU** les phrases confirmées
6. **Ajouter au dataset** pour entraînement 3-classes

Avantages de cette Approche

- ✅ **Semi-automatique** : Réduit le travail manuel de labeling
- ✅ **Basé sur l'incertitude du modèle** : Identifie les cas difficiles
- ✅ **Itératif** : Peut être répété pour améliorer le dataset

- ✅ **Calibré** : Les probabilités de Logistic Regression sont généralement bien calibrées

Limitations

- ⚠️ Nécessite un modèle binaire performant d'abord
- ⚠️ Les seuils doivent être ajustés selon les performances
- ⚠️ Validation manuelle recommandée pour éviter les faux ambigus

---

8. Final 3-Class Model Training

Pipeline Complet

Une fois la classe AMBIGU ajoutée au dataset, le pipeline devient :

1. Preprocessing (identique)

- Normalisation minimale
- Split train/validation/test avec stratification sur 3 classes

2. Vectorization (identique)

- TF-IDF avec mêmes paramètres
- Réentraînement sur le nouveau dataset 3-classes

3. Entraînement du Modèle

Options de modèles :

A. Logistic Regression (Recommandé)

- Multi-class avec `multi_class='multinomial'`
- Probabilités calibrées pour les 3 classes
- Interprétabilité maintenue

B. Linear SVM

- Multi-class avec `OneVsRest` ou `OneVsOne`
- Nécessite calibration pour probabilités

C. Modèles plus avancés (Phase future)

- Random Forest, Gradient Boosting
- Transformers (CamemBERT, FlauBERT)

4. Évaluation Spécifique à 3 Classes

Métriques importantes :

- **F1-Score par classe** : VALID, INVALID, AMBIGU
- **F1-Score macro** : Moyenne non pondérée (important pour AMBIGU minoritaire)
- **Matrice de confusion 3x3** : Visualiser toutes les erreurs
- **Recall pour AMBIGU** : S'assurer qu'on détecte bien les cas ambigus
- **Précision pour AMBIGU** : Éviter de sur-classifier comme AMBIGU

Analyse des erreurs :

- VALID classé comme AMBIGU : Acceptable (demande de clarification)
- INVALID classé comme AMBIGU : Acceptable (demande de clarification)
- AMBIGU classé comme VALID/INVALID : À minimiser (perte d'information)

5. Export du Modèle

Artefacts à sauvegarder :

- Modèle entraîné (`.pkl` ou `.joblib`)

- Vectorizer TF-IDF (`.pkl`)
- Métadonnées :
  - Version du modèle
  - Date d'entraînement
  - Métriques de performance
  - Distribution des classes d'entraînement
  - Paramètres d'entraînement

Structure de sauvegarde :

```
models/
  └── baseline/
      └── logistic_regression_3class_2024-01-15_v1.pkl
      └── tfidf_vectorizer_2024-01-15.pkl
      └── metadata.json
```

---

9. Conclusion

Résumé du Pipeline Complet

1. **Génération du Dataset** (50k-100k phrases)
   - 70% INVALID, 30% VALID (phase binaire)
   - Diversité maximale, fautes réalistes, variations linguistiques
2. **Preprocessing Minimal**
   - Lowercase, nettoyage espaces, préservation de la diversité
3. **Vectorization TF-IDF**
   - N-grams (1,2), max_features=50k
   - Baseline robuste et rapide
4. **Entraînement Baseline Binaire**
   - Logistic Regression (recommandé)
   - Linear SVM, Naive Bayes (comparaison)
   - Métriques : Accuracy, F1, Recall INVALID, Precision VALID
5. **Détection Automatique d'Ambiguïté**
   - Utilisation des probabilités (zone [0.40, 0.60])
   - Bootstrap des échantillons AMBIGU
6. **Entraînement Modèle 3 Classes**
   - Distribution : 60% INVALID, 30% VALID, 10% AMBIGU
   - Réentraînement TF-IDF + Logistic Regression
   - Évaluation spécifique 3 classes
7. **Export et Déploiement**
   - Sauvegarde modèle + vectorizer + métadonnées
   - Prêt pour intégration production

Prochaines Étapes et Améliorations Futures

Court Terme

- ✅ Valider le pipeline baseline binaire
- ✅ Atteindre > 85% F1-score sur validation

- ✅ Identifier et corriger les patterns d'erreurs

Moyen Terme

- 🔄 Ajouter la classe AMBIGU
- 🔄 Optimiser les hyperparamètres
- 🔄 Améliorer le dataset avec les cas problématiques identifiés

Long Terme

- 🚀 **Fine-tuning de CamemBERT/FlauBERT**
  - Modèles pré-entraînés en français
  - Meilleure compréhension contextuelle
  - Performance supérieure attendue
- 🚀 **Expansion du Dataset**
  - Augmenter à 100k+ phrases
  - Ajouter plus de variations linguistiques
  - Inclure des exemples de production (feedback utilisateurs)
- 🚀 **Extension Multilingue**
  - Ajouter d'autres langues (anglais, espagnol)
  - Modèles multilingues ou séparés par langue
  - Gestion du code-switching
- 🚀 **Optimisation Production**
  - Quantization du modèle
  - Optimisation de la latence
  - Pipeline de monitoring et feedback

Principes Clés

- **Diversité avant tout** : Le dataset doit refléter la réalité utilisateur
- **Baseline solide** : Valider avec des modèles simples avant d'aller vers le complexe
- **Itération continue** : Améliorer le dataset basé sur les erreurs du modèle
- **Minimal preprocessing** : Préserver la diversité naturelle du langage
- **Métriques équilibrées** : Ne pas optimiser seulement l'accuracy globale

---

Document Version: 1.0
Last Updated: 2024-01-15
Status: Active Development